🎤 Speech Entrega Final — SGR-DIGI

---

1. ¿Qué problema resolvemos?

El problema real: Las Pymes argentinas no acceden al crédito

Las **Sociedades de Garantía Recíproca (SGR)** son entidades que otorgan avales a Pymes para que puedan acceder a financiamiento bancario. Hoy, el proceso de emisión y gestión de esos avales es:

- 📄 **Papelero y lento**: Los avales se emiten como documentos PDF, se firman a mano, se envían por mail. Un aval puede tardar días en validarse.
- 🔍 **Opaco**: No hay forma transparente de verificar si un aval es auténtico, si sigue vigente, o si ya fue utilizado como respaldo de un crédito.
- 🔒 **Ilíquido**: Una vez emitido, el aval queda "trabado". La Pyme no puede fraccionar su garantía ni comercializarla. Si una Pyme tiene un aval de $55.000 USD en soja pero solo necesita $20.000 en financiamiento, no puede hacer nada con los $35.000 restantes.
- 🏦 **Sin acceso a DeFi**: Los avales respaldados en commodities (soja, trigo, maíz) son activos con valor real, pero están completamente desconectados del ecosistema financiero descentralizado. No se pueden usar como colateral para préstamos on-chain.

> ***En resumen:*** *Hay miles de millones en commodities agropecuarios argentinos que respaldan avales, pero esa riqueza real está atrapada en papeles que nadie puede verificar, fraccionar ni usar de forma eficiente.*

---

2. ¿Cómo lo resolvemos?

SGR-DIGI: Tokenización de Avales sobre Stellar

Construimos una plataforma fullstack que **digitaliza y tokeniza** el ciclo completo de las garantías recíprocas usando blockchain e inteligencia artificial.

Flujo end-to-end (tal como funciona en la plataforma):

```
📋 El operador ingresa a la plataforma (Dashboard) y ve KPIs, procesos activos y certificados
emitidos
       ↓
📤 Desde "New Guarantee", sube un documento del commodity (PDF, JPG o PNG, hasta 20 MB)
       ↓
🤖 La IA (Google Gemini) analiza el documento y extrae datos estructurados:
   tipo de instrumento, cantidad, valor en USD, y nivel de riesgo (BAJO/MEDIO/ALTO)
       ↓
✏️ El operador revisa y corrige los datos extraídos en un formulario interactivo
   (DataCorrectionForm — datos personales, información del aval, datos financieros)
       ↓
📊 Se calcula el aval: valor base × factor de riesgo × multiplicador = monto final
       ↓
✅ Modal de confirmación muestra resumen completo: documento, empresa, cálculo y red
blockchain
       ↓
⛓️ Al confirmar, el backend invoca el Smart Contract "GuaranteeToken" y mintea tokens AURA on-
chain
```

```
        ↓
🔗 Se recibe el txHash y un link directo a Stellar Explorer para verificación instantánea
        ↓
🏪 Desde el Marketplace, el operador puede crear ofertas de venta de fracciones del aval
        ↓
🤝 Otros inversores ven las ofertas activas y compran fracciones (peer-to-peer, on-chain real)
        ↓
🏦 Desde BLEND Protocol Lending, los tokens AURA se usan como colateral para pedir prestado
XLM
        ↓
💰 La Pyme accede a liquidez inmediata contra sus avales tokenizados
```

¿Qué logramos concretamente?

Problema	Solución SGR-DIGI
Avales en papel	**Tokens digitales** en la blockchain de Stellar (AURA)
Verificación lenta	**txHash + Stellar Explorer** verificable instantáneamente
Validación manual	**IA con Gemini** — extracción automática de datos + clasificación de riesgo
Revisión humana lenta	**Formulario de corrección** — el operador valida y ajusta antes de emitir
Aval ilíquido	**Marketplace P2P on-chain** — se pueden fraccionar y vender partes del aval
Sin acceso DeFi	**Integración con BLEND Protocol** — avales como colateral para préstamos
Proceso opaco	**Registro inmutable** — cada operación queda grabada on-chain
Sin visibilidad	**Dashboard con KPIs** — garantías emitidas, volumen, tiempos, procesos activos

---

3. La Plataforma — Lo que construimos

🎨 Frontend — React + TypeScript + Vite + Tailwind CSS

La plataforma cuenta con **9 páginas** organizadas bajo un layout profesional con sidebar de navegación y header, conectadas a la blockchain de Stellar Testnet:

Páginas Funcionales (core del MVP):

1. **Dashboard** (`/dashboard`) — Panel principal con:
    - 4 KPIs: Garantías emitidas, Volumen garantizado, Tiempo promedio de emisión, Procesos activos
    - Tabla de procesos en curso con estados visuales (Document Uploaded → AI Processing → Human Validation → Registered On-Chain → Certificate Issued)
    - Certificados emitidos con hash on-chain y red Stellar Testnet
    - Log de actividad en tiempo real
    - Widget de nueva garantía y flujo de workflow visual
2. **New Guarantee** (`/guarantee`) — Flujo completo en 2 pasos:
    - **Paso 1**: Upload con drag & drop (PDF, JPG, PNG hasta 20 MB), validación de formato y tamaño
    - **Paso 2**: Formulario de corrección con datos extraídos por IA, organizados en secciones colapsables (Datos Personales, Información de Aval, Datos Financieros)

    - **Modal de confirmación**: Resumen del documento, empresa beneficiaria, cálculo del aval (valor base × riesgo × multiplicador), datos blockchain. Advertencia de acción irreversible.
    - **Emisión real**: POST al backend → smart contract → txHash + link a Stellar Explorer

3. **Marketplace** (`/marketplace`) — Mercado descentralizado:
    - **Crear ofertas**: Seleccionar garantía activa, definir cantidad de tokens AURA y precio en XLM
    - **Comprar fracciones**: Ver ofertas activas por garantía, comprar con cantidad personalizada
    - **Todo on-chain**: Cada operación invoca el contrato Marketplace en Stellar Testnet
4. **BLEND Protocol Lending** (`/lending`) — Interfaz DeFi:
    - Panel con 3 métricas: Colateral total (AURA), Deuda total (XLM), Health Factor
    - Barra de progreso de uso del límite de préstamo
    - Sección Supply: depositar AURA como colateral con simulación en tiempo real
    - Sección Borrow: pedir prestado XLM hasta el 70% del valor del colateral
    - Simulación interactiva del nuevo Health Factor antes de confirmar
    - Alertas de riesgo de liquidación cuando el Health Factor baja de 1.05
    - Botón de repago total de deuda

> ***Nota sobre BLEND**: La página de Lending funciona como **mock interactivo del MVP** — simula toda la mecánica del protocolo (supply, borrow, health factor, liquidation warnings) en el frontend. El backend tiene el `LendingService` completo con integración real a BLEND Protocol, listo para activarse con fondos de liquidez en la pool.*

Páginas Placeholder (roadmap futuro):

5. **Processes** (`/processes`) — Gestión detallada de procesos
6. **Certificates** (`/certificates`) — Descarga de certificados (QR + hash)
7. **Blockchain Explorer** (`/explorer`) — Explorador de transacciones on-chain
8. **Companies** (`/companies`) — Perfiles de empresas y su historial de garantías
9. **Settings** (`/settings`) — Configuración de la plataforma

Componentes Reutilizables:

- `Layout` — Sidebar con navegación + Header + indicador de estado (Stellar Testnet online)
- `DataCorrectionForm` — Formulario inteligente con secciones colapsables, validación y agrupación automática de campos
- `EmissionConfirmationModal` — Modal con resumen completo y resultado de la transacción
- `KPICard` , `ProcessTable` , `CertificateCard` , `WorkflowSteps` , `ActivityLog` , `NewGuaranteeWidget` — Componentes del dashboard
- `Modal` — Componente base reutilizable con overlay, animaciones y manejo de teclado

---

🔗 Backend — NestJS + TypeScript

Arquitectura modular con **4 módulos funcionales**:

`ExtractionModule` — Procesamiento de documentos con IA

- **`ExtractionController`** : 4 endpoints
    - `POST /extract` — Extracción desde texto plano
    - `POST /extract/upload` — Upload de archivos (PDF, JPG, PNG) con Multer
    - `GET /extract/mock/:id` — Documentos mock para testing (DOC-001, DOC-002, DOC-003)
    - `GET /extract/mock` — Listar documentos mock disponibles

- `LlmService` : Integración con **Google Gemini** (`gemini-2.0-flash`)
  - System prompt especializado en análisis SGR argentino
  - Extrae: tipo de documento, cantidad, valor en USD, flag de riesgo
  - Fallback con heurísticas (regex) si Gemini no está disponible
- `FileService` : Procesamiento de archivos
  - Soporte para PDF (pdf-parse), imágenes (JPG, PNG) y texto plano
  - Validación de tipos MIME

`StellarModule` **— Core blockchain**

- `StellarController` : 7 endpoints
  - `GET /stellar/health` — Health check de la red Stellar
  - `POST /guarantee` — Crear garantía (mintea tokens AURA on-chain)
  - `GET /guarantee` — Listar todas las garantías
  - `GET /guarantee/:id` — Detalle de una garantía
  - `POST /guarantee/:id/retry` — Reintentar emisión fallida
  - `POST /marketplace/offer` — Crear oferta de venta
  - `POST /marketplace/buy` — Comprar tokens de una oferta
  - `GET /marketplace/offers/:guaranteeId` — Ver ofertas por garantía
- `StellarService` : ~500 líneas de lógica blockchain
  - Comunicación con Soroban RPC via `@stellar/stellar-sdk`
  - Flujo transaccional: Simulación → Firma → Envío → Polling async hasta confirmación
  - Gestión de nonces, fees estimados, y manejo de errores
- `OperationsRepository` : Persistencia JSON con idempotencia

`LendingModule` **— DeFi (BLEND Protocol)**

- `LendingController` : 5 endpoints
  - `POST /lending/supply` — Depositar AURA como colateral
  - `POST /lending/borrow` — Pedir prestado XLM
  - `POST /lending/repay` — Repagar deuda
  - `GET /lending/position` — Consultar posición (colateral, deuda, health factor)
  - `GET /lending/status` — Estado de configuración (pool ID, oracle, backstop)
- `LendingService` : ~400 líneas con integración real a BLEND Protocol
  - Supply collateral, borrow, repay
  - Cálculo de health factor on-chain
  - Configurado con pool, oracle y backstop desplegados

`AvalModule` **— Lógica de negocio de avales**

- Módulo dedicado para la lógica específica del dominio SGR

🔗 Smart Contracts — Stellar (Soroban / Rust → WASM)

3 contratos desplegados en Stellar Testnet, escritos en **Rust** y compilados a **WebAssembly**:

1. `GuaranteeToken` — Token SEP-41 que representa los avales
   - Funciones: `mint` , `transfer` , `balance` , `burn` , `allowance` , `approve`
   - Cada token AURA está respaldado por un commodity real
   - Cumple el estándar SEP-41 (equivalente a ERC-20 en Ethereum)
2. `Marketplace` — Mercado descentralizado peer-to-peer

- Funciones: `create_offer`, `buy_tokens`, `get_offers`
- Permite crear ofertas de venta y ejecutar compras de fracciones de avales
- Manejo de escrow automático durante la transacción

3. **`CommodityOracle`** — Oráculo de precios on-chain
   - Alimenta al protocolo BLEND con el precio en USD de los tokens AURA y XLM
   - Permite calcular la salud financiera de los préstamos

🤖 Inteligencia Artificial — Google Gemini

- **Modelo**: `gemini-2.0-flash` via `@google/generative-ai`
- **System prompt** especializado en análisis de documentos SGR argentinos
- **Output estructurado**: Tipo (FACTURA, WARRANT, PAGARÉ, CERTIFICADO_DEPOSITO), cantidad, valor USD, nivel de riesgo
- **Fallback inteligente**: Si Gemini no está disponible, usa extracción por regex/heurísticas
- **Conversión de moneda**: Convierte ARS a USD usando tipo de cambio del documento o referencia de mercado

---

4. Contratos Desplegados (verificables en Stellar Explorer)

Contrato	ID en Testnet
GuaranteeToken (AURA)	`CC4HUPWEQCRRME4AQKND72QFDMLVYD3TX5DUJ2MCU5KN25UUFWWZDTTH`
Marketplace	`CCX37O22MXY3CMZIU3KYCCQQFO6NIJYYT4HVIOUZ2H22J4QHPAAIK7BN`
Commodity Oracle	`CA3Q57COKCOX3FUQGDWDBLY7RFITECDQ3RJIABSFJJLHIREWFNLJVN3U`
BLEND Lending Pool	`CBFFQS7ZAICT3Z46VYDZUBLOKBGJDM2FWCNAMILCBIPVURCNWXPCQM5H`

> 🔗 *Verificar en:* `https://stellar.expert/explorer/testnet/contract/<ID>`

---

5. Stack Tecnológico Completo

Capa	Tecnología	Detalle
Frontend	React 19 + TypeScript + Vite 8	SPA con 9 páginas, routing con React Router v7
Styling	Tailwind CSS 4	Utilidades CSS, diseño responsive, glassmorphism
Iconografía	Lucide React	Librería de iconos consistente en toda la UI
Backend	NestJS 11 + TypeScript	4 módulos, inyección de dependencias, guards
IA	Google Gemini 2.0 Flash	`@google/generative-ai`, extracción de documentos
Procesamiento	pdf-parse, Multer	Upload y parsing de PDF/imágenes
Blockchain	Stellar Testnet + Soroban	`@stellar/stellar-sdk` v14.6
Smart Contracts	Rust → WebAssembly	3 contratos desplegados (SEP-41, Marketplace, Oracle)
DeFi	BLEND Protocol	Pool propio con supply/borrow/repay

Persistencia	JSON local	Repositorio con idempotencia y reintentos

---

6. Diagrama de Arquitectura

```
┌──────────────────────────────────────────────────────────────────────┐
│                    FRONTEND (React 19 + Vite 8 + Tailwind 4)         │
│                                                                      │
│  Dashboard │ New Guarantee │ Marketplace │ BLEND Lending              │
│  (KPIs,    │ (Upload +     │ (Create     │ (Supply AURA,              │
│   Procesos,│  IA + Emit)   │  Offer +    │  Borrow XLM,               │
│   Certs)   │               │  Buy P2P)   │  Health Factor)            │
│                                                                      │
│  Processes │ Certificates │ Explorer │ Companies │ Settings           │
│  (roadmap) │ (roadmap)    │ (roadmap)│ (roadmap) │ (roadmap)          │
└──────────────────────────┬──────────────────────────────────────────┘
                           │ REST API (HTTP → localhost:3000)
                           │ Endpoints: /extract/* /guarantee/*
                           │            /marketplace/* /lending/*
┌──────────────────────────▼──────────────────────────────────────────┐
│                       BACKEND (NestJS 11)                            │
│                                                                      │
│  ExtractionModule          │ StellarModule                           │
│  ├─ LlmService (Gemini)    │ ├─ StellarService (@stellar/sdk)        │
│  ├─ FileService (PDF/IMG)  │ ├─ OperationsRepository (JSON)          │
│  └─ ExtractionController   │ └─ StellarController                    │
│                            │   ├─ /guarantee (CRUD)                  │
│  AvalModule                │   └─ /marketplace (offer/buy)           │
│  └─ Lógica de negocio SGR  │                                         │
│                            │ LendingModule                           │
│                            │ ├─ LendingService (BLEND SDK)           │
│                            │ └─ LendingController                    │
│                            │   └─ /lending (supply/borrow/repay)     │
└──────────────────────────┬──────────────────────────────────────────┘
                           │ Soroban RPC (Stellar Testnet)
┌──────────────────────────▼──────────────────────────────────────────┐
│                    STELLAR BLOCKCHAIN (Testnet)                      │
│                                                                      │
│  ┌─────────────────┐  ┌─────────────────┐  ┌─────────────────┐      │
│  │ GuaranteeToken  │  │   Marketplace   │  │ CommodityOracle │      │
│  │ (Token AURA)    │──│ (Compraventa)   │  │ (Precios USD)   │      │
│  │ SEP-41          │  │ P2P             │  │                 │      │
│  └─────────────────┘  └─────────────────┘  └────────┬────────┘      │
│                                                     │               │
│  ┌──────────────────────────────────────────────────▼──────────┐    │
│  │              BLEND Protocol (DeFi Lending)                  │    │
│  │  Pool Factory → AURA Lending Pool → Backstop (Seguro)       │    │
│  │  Supply AURA (colateral) → Borrow XLM → Repay + intereses   │    │
│  └──────────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────┘
```

---

7. Demo — Flujo que se puede mostrar en vivo

Secuencia recomendada de demostración:

1. **Dashboard** → Mostrar KPIs, procesos en curso, certificados emitidos, flujo de workflow
2. **New Guarantee** → Subir un PDF de test → ver extracción IA → corregir datos → confirmar emisión → ver txHash en Stellar Explorer
3. **Marketplace** → Seleccionar la garantía recién creada → crear oferta de venta (ej: 1000 tokens AURA a 10 XLM) → ver la oferta publicada
4. **BLEND Lending** → Simular depositar 5000 AURA → ver borrow limit → pedir prestado XLM → mostrar health factor bajando → repagar
5. **Stellar Explorer** → Abrir `stellar.expert/explorer/testnet/contract/<ID>` y verificar el contrato y las transacciones on-chain

> ***Tips para la demo***:
>
> - *Tener el backend corriendo (* `npm run start:dev` *en* `/backend` *)*
> - *Usar el Mock Document DOC-001 si la IA tarda en responder (* `GET /extract/mock/DOC-001` *)*
> - *Mostrar el sidebar con "Stellar Testnet online" como indicador de conectividad*

---

8. Frase de cierre

> *"SGR-DIGI transforma avales de commodities agropecuarios — que hoy son papeles estáticos — en activos digitales verificables, fraccionables y colateralizables sobre la blockchain de Stellar. Conectamos la economía real argentina con las finanzas descentralizadas, dándole a las Pymes herramientas que hoy solo tienen las grandes corporaciones."*

---

9. Datos técnicos clave para Q&A

- **¿Por qué Stellar?** Transacciones en ~5 segundos, fees menores a $0.01, red enfocada en pagos y activos financieros. Soroban es su VM de smart contracts (Rust → WASM).
- **¿Por qué BLEND?** Es el protocolo de lending nativo de Stellar/Soroban. Open-source, permissionless, con backstop de seguridad colectivo. Permite crear pools de lending propios.
- **¿Qué es SEP-41?** Es el estándar de tokens en Soroban (equivalente a ERC-20 en Ethereum). Nuestro token AURA cumple con él, lo que lo hace interoperable con cualquier DEX o protocolo DeFi de Stellar.
- **¿Cuántos contratos tiene?** 3 propios (GuaranteeToken, Marketplace, Oracle) + 1 pool creado en BLEND Protocol = 4 contratos on-chain.
- **¿Funciona en producción?** Está desplegado en Testnet. Para Mainnet se necesitaría auditoría de contratos, oráculo de precios real (no hardcoded), fondos de liquidez reales, y wallet de usuario (Freighter).
- **¿Qué hace la IA?** Google Gemini analiza los documentos del commodity y extrae automáticamente tipo, cantidad, valor USD y nivel de riesgo. Si Gemini no está disponible, hay un fallback con extracción por regex.
- **¿El Marketplace es real o mock?** El Marketplace es **100% real** — cada oferta y compra es una transacción on-chain en el contrato Marketplace. El frontend se comunica con el backend que invoca el smart contract vía Soroban RPC.
- **¿Y el Lending?** El frontend de Lending funciona como **mock interactivo** para el MVP (simula la mecánica localmente). El backend tiene la integración real completa con BLEND Protocol (supply, borrow, repay, position, health factor). Se puede activar cuando haya fondos de liquidez en la pool.
- **¿Cuántas páginas tiene el frontend?** 9 páginas: 4 funcionales (Dashboard, New Guarantee, Marketplace, Lending) y 5 placeholder para roadmap futuro (Processes, Certificates, Explorer, Companies, Settings).

- **¿Qué frameworks usan?** Frontend: React 19 + Vite 8 + Tailwind 4 + React Router 7. Backend: NestJS 11 + TypeScript. Contratos: Rust + Soroban SDK.
- **¿Cómo se comunican frontend y backend?** REST API sobre HTTP. El frontend hace fetch a `localhost:3000` (backend) y `localhost:8000` (extraction service). El backend habla con Stellar Testnet vía Soroban RPC.